# Unit → 2

→ Inheritance

- When we construct a new class from an existing class in such a way that new class accesses all the features & properties of existing class
- In java, 'extends' keyword is used to perform inheritance
- It provides code reusability.
- We can't access private members of class through inheritence.
- A subclass contains all the features of super class. so, we should create the object of subclass.
- Method overriding is only possible through inheritence

```
class A {
  ↳ superclass
}
  ↳ subclass
class B extends A {
}
```

→ Types of inheritance

1. Single / Simple inheritance

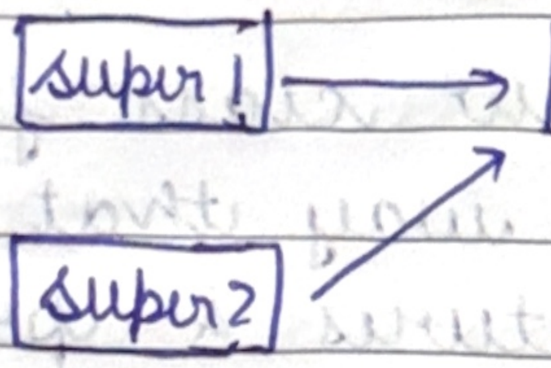



2. Multilevel inheritance



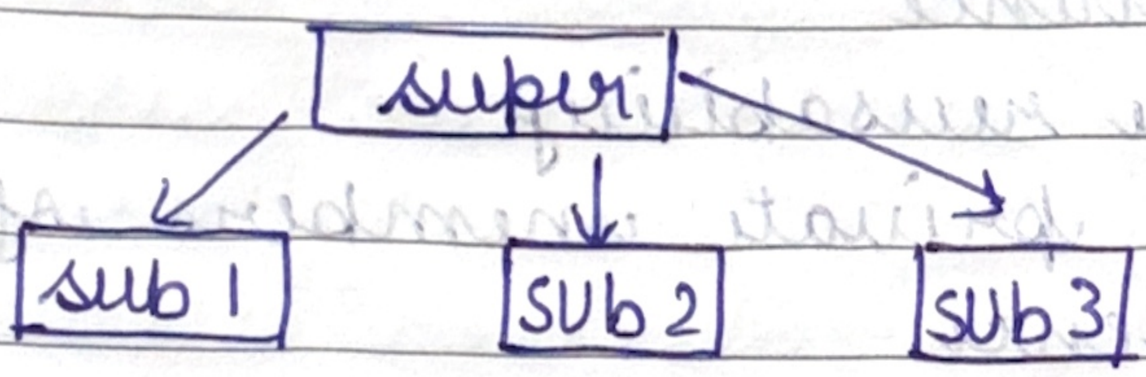
3. Multiple inheritance

- Not supported in java via classes




- done using interfaces

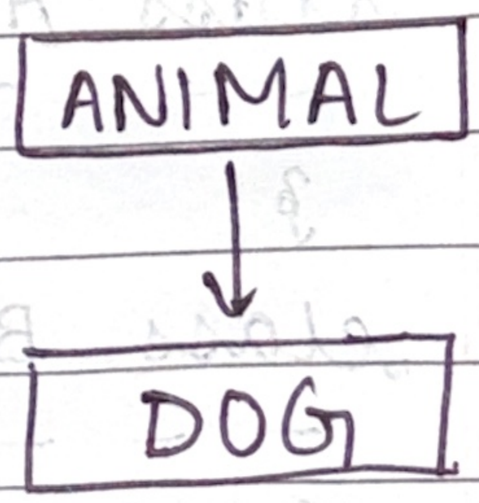
4. Hierarchial inheritance




→ Single Inheritance

- A single sub class inherits from a single super class

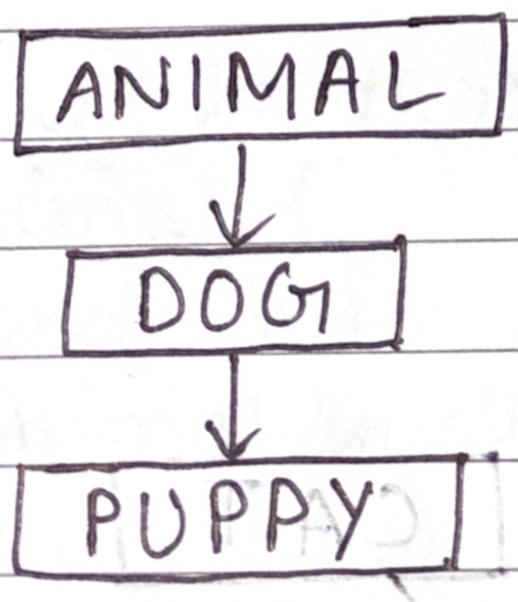
```
class Animal {
void eat(){
sout ("eats");
  }
}
class Dog extends Animal {
 void bark(){
sout ("barks");
  }
}
public class Main {
public static void main () {
 Dog labra = new Dog ();
labra.eat ();
labra.bark ();
  } }
```



→ Multilevel Inheritance

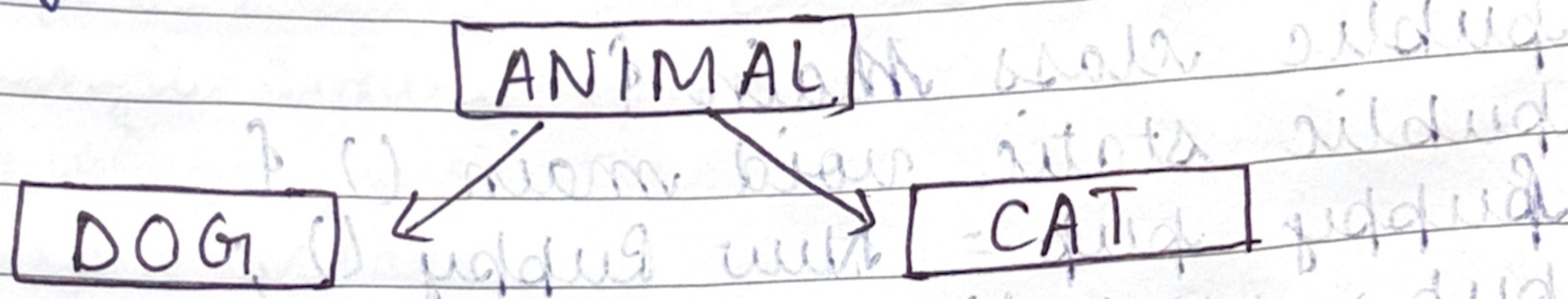
• Forms a sequential chain of inheritance

```
class Animal {
void eat () {
sout ("eats");
} }
class Dog extends Animal {
void bark () {
sout ("barks");
} }
class Puppy extends Dog {
void wwep () {
sout ("cry aagaya :(");
} }
public class Main {
public static void main () {
Puppy pup = new Puppy ();
pup. wwep ();
pup. bark ();
pup. eat ();
} }
```



→ Hierarichal Inheritance

```
class Animal {
void eats () {
 souts ("eats");
  } }
class Dog extends Animal {
void barks () {
sout ("barking");
  } }
class Cat extends Animal {
void meow () {
sout ("meow meow");
 }
 }
```




→ Hybrid Inheritance
uses multiple types altogether

→ super keyword
Reference variable to call constructors / methods / variables of parent class in the child class in case of overriding or overloading

1. variable (same names)

```
class Animal {
string color = "White";
}
class Dog extends Animal {
string color = "Black";
public void display colors () {
sout ("Dog's color :" + color);
sout ("Animal's color :" + super.color);
}
}
public class Main {
public static void main () {
Dog dog = new Dog ();
dog.display colors ();
}
```

2. In case of method overriding

```
class Animal {
void sound () {
sout ("Animal makes generic sound");
}}
class Dog extends Animal {
@override → child class gives its own defination of method defined in parent class using same name & paramiters
void sound () {
sout ("Dog barks : woof woof");
}
```

```
public void callBothSounds() {
super.sound(); → defined in parent class
sound(); → defined in child class
}
}
public class Main {
public static void main() {
Dog dog = new Dog();
dog.callBothSounds();
}}
```

3. To use data defined in parent class constructor
   - Child class must invoke it's parent class constructors to ensure proper initialization of parent fields.

```
class Animal {
String name;
public Animal(String name) {
this.name = name;
sout("Animal constructor called for :" + name);
}}
class Dog extends Animal {
String breed;
public Dog(String breed, String name) {
this.breed = breed; super(name);
sout("Dog constructor called for :" + breed);
}}
public class Main {
public static void main() {
Dog dog = new Dog("Labra", "Buddy");
}}
```

→ Final Keyword in JAVA

- It's a non access modifier used to apply restrictions on variables, methods & classes
- It is used to prevent inheritance and overriding.

1. Prevents method overriding
   - When you mark a method as final, it means the method implementation is complete and locked down
   - A subclass can't override a final method

eg :-

```
class vehicle {
public final void start () {
sout ("engine started");
}}
class car extends vehicle {
@Override
void start () { - - - } X throws an error
```

2. Prevents Inheritance
   - On marking a class as final, it can be extended by any other class
   - If a class is declared final, its methods also become final

→ Polymorphism

- Polymorphism means many forms
- It allows a single method name to behave differently based on the objects involved.

Polymorphism

- Compile time (Static Binding) uses method overloading
- Run time (Dynamic Binding) uses method overriding

1. Compile time Polymorphism

- Class has multiple methods with same name but different parameters (list)

eg :-

```
class Calculator {
int add (int a, int b) {
  return a+b;
   }
int add (int a, int b, int c) {
   return a+b+c;
   }
int add (int a, int b, int c, int d) {
   return a+b+c+d;
   }}
public class Main {
public static void main () {
 Calculator calci = new Calculator ();
sout (Calc. add (10, 20));
sout ( Calc. add ( 3, 4, 5, 6));
 }}
```

2. Runtime Polymorphism

- occurs when subclass provides a specific implementation of a method already defined in it's parent class.

eg :-

```
class Animal {
void sound () {
sout ("roar");
  }}
class Dog extends Animal {
@override
void sound () {
sout ("barks");
 }}
class Cat extends Animal {
@override
void sound () {
 sout ("meows");
}}
public class Main {
 public static void main () {
Animal a1 = new Dog (); → calls the dog's overridden method
Animal a2 = new Cat ();
}
}
```

↓ calls cat's overridden method

→ Abstract classes & methods

- Declared using abstract keyword
- Abstract classes serve as blueprint for class that extends it
- Abstract methods are just declared but not defined in parent class & are

later defined in the subclass.

```
abstract class Vehicle {
abstract void start engine ();
```

↓ abstract method

```
class car extends Vehicle {
@ override
void start engine () {
sout ("vroom vroom");
  }
```

↳ defined in sub class

→ Interfaces

- while both interfaces and abstract classes are used to achieve abstraction in JAVA, there are some key diff :-

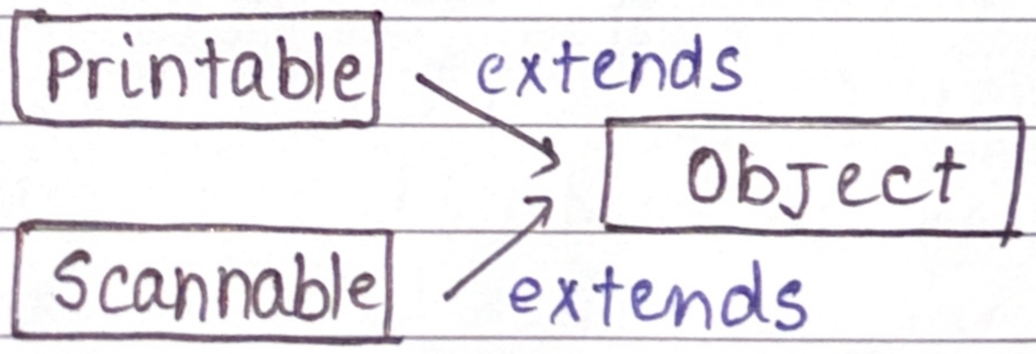
1. Abstract class
   - A class can extend only 1 abstract class
   - It can have instance, non-final variables
   - can have constructors
   - can have both abstract & fully implemented methods
2. Interface
   - can have multiple inheritance (implements)
   - fields are compulsarily public + static + final
   - can't have constructors (can't be instantiated)
   - can have abstract methods only

```
interface Flyable {
void fly();
}
class Bird implements Flyable {
@ override
public void fly {
sout ("soars through sky");
}
```

→ Implementation of interfaces
- use implement keyword
- class must override & provide bodies for all abstract methods declared in interface
- overridden methods of implementing class must be public

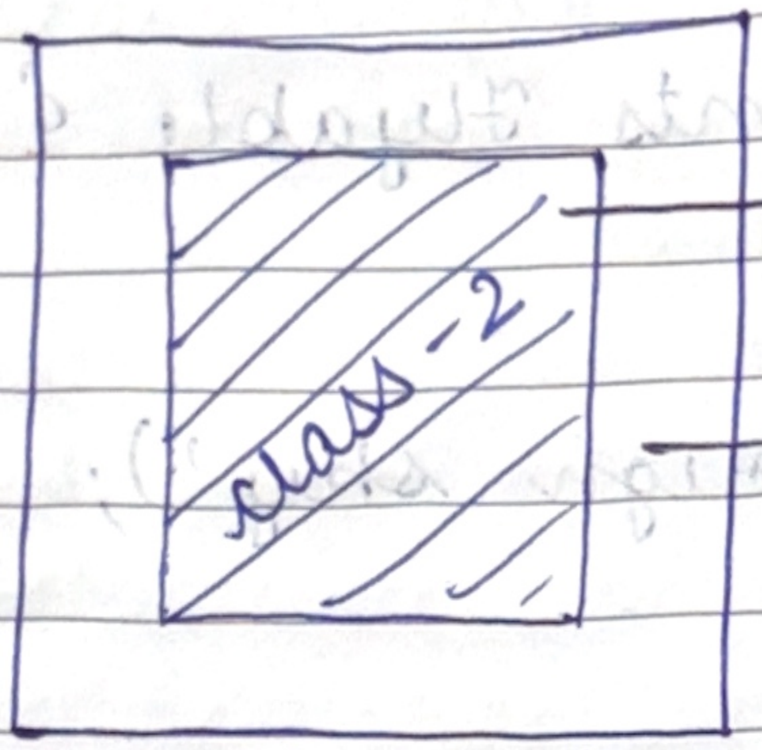
→ Inheritance of Interfaces (extension) → Jab koi class ek child interface ko implement kregi to uo parent + child dono ke abs. methods ko define kregi

interface Animal
↓ extends
interface Mammal
↓ implements
class DOg




Multiple inheritance is possible

→ Nested Classes
class inside another class




```
class Outer {
private String msg = "Hii from outer";

class Inner {
public void display () {
sout (msg);
}
}
public class Main {
public static void main () {
Outer out = new Outer ();

Outer. Inner in = out . new Inner ();
in . display ();
}
}
```

→ Package

A package is a mechanism used to group related classes, interfaces and sub packages together

1. Built in Packages
   - Provided by the JAVA API
   - eg → java.util, java.io, java.net etc.
2. User defined Packages
   - custom packages created by developers to organize their own applications

→ for eg :-

```
package mathutil;
public class calculator {
public int add (int a, int b) {
return a+b;
}
}
```

→ Importing a package

```
import java.util.Scanner;
import mathutil.*;
```